trick

1. 一个数末尾 0 的个数 : $min(c_2, c_5)$ ，其中 c_2, c_5 分别表示 x 中含有 $2, 5$ 因子的个数

   要求区间，或者单个数满足末尾 0 的个数为 k 的数，要想到二分，双指针，前缀和

2. 遇到多个求和符号或者求所有子区间的什么价值和：$\sum\sum$ 考虑单个元素对答案(区间)的贡献

3. 注意每次求 gcd 都会让 x 不变或者至少使得 $x/2$，故一个以 i 为结尾(开头)的子区间不同的 gcd 个数是 log 级别

4. 看到 n^2 要想到 $\sqrt{n}$ 的枚举，降低复杂度

5. 越 $|$ 越大，越 $\&$ 越小

   一个以 i 为结尾(开头)子区间的 $|$ 不同的或运算结果是 log 级别的，因为最多 $log_2(x)$ 位的 1

   $\&$ 的特点，当一位为 0，这一位在不同的做 $\&$ 都还是 0 了

6. 要注意是否有均摊的性质(特别是线段树题)

7. 对于计算 $a \times b = k$ 这样的对数，要想到 a, b 都要是 k 的因子，从而降低枚举等等

   对于从集合里面选一些数问你：$gcd(a_i, a_j, \ldots, a_k)$或者 $lcm(a_i, a_j, \ldots, a_k)$ 最大或者最小值，要想到枚举 gcd 或者 lcm 的取值，然后转化为判断是否存在倍数和因子的问题

8. 对于一些正着求不好求的问题，要想到正难则反，即尝试求其反集。
   - 求 $<$ ，可以转为求 $\geq$
   - 求 $>$ ，可以转为求 $\leq$
   - 求 $= k$ ，可以转为求 $(\geq k) - (\geq k+1)$，或者 $(\leq k) - (\leq k-1)$
   - 求 $\sum_{i=x_1}^{x_2}\sum_{j=y_1}^{y_2}$ ，可以考虑二维前缀和的求法，先令 $f(x, y) = \sum_{i=1}^{x}\sum_{j=1}^{y}$

   故：$g(x_1, x_2, y_1, y_2) = f(x_2, y_2) - f(x_2, y_1 - 1) - f(x_1 - 1, y_2) + f(x_1 - 1, y_1 - 1)$
   - 求在区间 $[l, r]$ 中满足 $\gcd(p, x) = 1$ 的 x 的个数，我们可以转成求 $\gcd(p, x) \neq 1$，这个问题可以用**容斥求解。**

9. 遇到求最小值，最大值，最小值最大，最大值最小，记得考虑二分答案

   当发现具有单调性，二分性，一定要想到二分

   二分的时候，注意定义$check$，有些题目是定义为 $= mid$，有些则是 $\geq mid, \leq mid$

   这一点很重要，这决定了我们是否具有二分性质，以及 $check$ 的写法和含义

   求最值，如果没有二分性，考虑试试三分法，但难以证明凹凸性的时候，可以尝试打表，如果大部分数据都对，可以尝试尝试三分法

10. 对于具有单调性，求区间什么 $\leq k, \geq k, = k$ 一些计数题，要想到是否可以双指针，需要分析两个指针是否单调移动

    双指针也可以是：同向，反向，**对角线**等；双指针的时候，注意边界！！！！！（$l \leq r$）很重要

11. 关于什么互质，因子，质数问题，一定要想到 $x = p_1^{k_1} \times p_2^{k_2} \ldots \times p_n^{k_n}$ ，分解定理

12. 关于维护奇偶性，只要维护01即可，要想到可以用二进制的异或维护

13. 当发现 n 很小的时候，考虑是否是状压 dp 题，以及是否是容斥，dfs 爆搜题

14. 遇到对一个数进行若干次 $+k, -k$ 操作， 要想到 $x\%k$ 的余数是一定不会变的，要往同余方向想

15. 要对 $x\%p = x - \lfloor \frac{x}{p} \rfloor * p$ ，这样的转化敏感，当 $\%$ 不好处理的时候，可以转成下取整，反过来也同理

    同时对于 $\%$ 要注意， $x\%p \in [0, p-1]$ ，这里可能会考到鸽笼原理！！

    同时 $\%$ 可能还会考到周期性

    对于这种 $\lfloor \frac{n}{x} \rfloor$ ，要想到二分求 x，以及整除分块等问题，以及 $\lfloor \frac{n}{x} \rfloor$ 的不同取值最多只有 $O(\sqrt{n})$ 种

16. 对于期望概率题，先考虑 dp，如果不好 dp，考虑贡献法，或者转成求方案，变成计数问题

17. 对于匹配类的 dp，可以考虑 $dp[i][j]$ 表示前 i 个，还有 j 个未匹配

18. 对于计数类的 dp，要考虑是否要记录方案数和价值和，要同时维护，才好转移

19. 对于选取的某个集合的平均值为 x；

    即 $\frac{\sum a_i}{|S|} = x \leftrightarrow \sum a_i = |S| * x \leftrightarrow \sum (a_i - x) = 0$

20. 对于给你 m 对类似：$a_l + a_{l+1} + \ldots + a_r = x$，或者 $a_l \oplus a_{l+1} \oplus \ldots \oplus a_r = x$ 的关系

    我们可以利用前缀和思想，转化为：$s[r] - s[l-1] = x$ ，或者 $s[r] \oplus s[l-1] = x$

    给出这样的关系，我们可以看成 $s[r]$ 和 $s[l-1]$ 之间有一条双向边，转化为连通块问题

    这类题要想到并查集维护，判断矛盾关系，可以采用带权并查集，一旦某个连通块中的一个$s[t]$ 的值已经确定，和他在一个连通块的所有 $s[v]$ 都会被确定下来

21. 每一个不同的前缀和数组s 或者 差分数组 b 都对应着一个唯一不同的原数组 a

    故常常可以把求解不同的原数组个数，转化为求其不同的前缀和或者其不同的差分数组的个数

22. 经典贪心模型，有两种物品属性为 a_i, b_i 现在你要从中选 k 件物品属性是 a_i，其余 $n-k$ 件物品属性为 b_i

    问你最小/最大花费，我们可以先考虑都是 b_i，记 $sum = \sum b_i$ ，然后我们发现每换掉一个 b_i 其 sum 会变成：

    $sum - b_i + a_i$ ，故我们只要按 $a_i - b_i$ 排序，找前 k (大/小)

23. 对于一个数组，交换任意两个相邻的数：

    (1) 若数组所有数都不同，那么其逆序对个数的奇偶性一定会发生改变

    (2) 若存在相同的数，交换的两个数相同的话，奇偶性不变，否则也会发生改变

24. 看到多次询问删除 $[l, r]$ ，或者删除 i 影响之后的信息，且是不带修改的静态询问，我们要想到是否可以用前后缀分解的预处理

    ，然后询问的时候在合并 $pre[l-1]$ ，$suf[r+1]$，或者 $pre[i-1]$，$suf[i+1]$ 的信息

25. 给你一个序列 A ，问你序列中任意两个数异或的最小值是多少？

    结论：序列任意两个数异或最小值就是将数组排好序之后，所有相邻两个数的异或值的最小值。

    如果需要动态维护序列 A，以及这个最小值，对于增删改查，我们可以用两个 `multiset<int>a,b` 来解决

    a 维护序列 A，b 维护答案，那么 $*b.begin()$ 就是任意两个数异或的最小值

26. 要求 $(a_1, a_2, ..., a_n)$ 的公约数，等价于求 $gcd(a_1, a_2, .., a_n)$ 的公约数。

27. $((a_i \bigoplus x) \& (a_j \bigoplus x))_{max} \leftrightarrow (a_i \bigoplus a_j)_{min}$。

28. 欧拉降幂：

$$a^b \equiv \begin{cases} a^{b \bmod \phi(p)} & \gcd(a,p)=1 \\ a^b & \gcd(a,p) \neq 1, b < \phi(p) \\ a^{b \bmod \phi(p) + \phi(p)} & \gcd(a,p) \neq 1, b \geq \phi(p) \end{cases} \pmod p$$

29. 求最小环长度和数量（$n \leq 500$）

    考虑用 Floyd 求解，我们枚举环上编号最大的那个点 k 去把环分类。

    根据当枚举到 k 的时候 $g[i][j]$ 就表示 $i->j$ 的路径中，中间结点的编号范围为 $[1, k]$。

    我们只枚举到环中编号最大的顶点的时候统计，且我们枚举 (k, i) 这条边，使得他与 $g[i][k]$ 拼起来形成一个环。

```
const int N = 505, mod = 998244353;
LL f[N][N], g[N][N], cnt[N][N]; //cnt[i][j] 表示从i->j的最短路有几条
int n, m;
read(n), read(m);
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        cnt[i][j] = 0;
```

```
        if (i == j) g[i][j] = 0;
        else g[i][j] = 1e15;
    }
}
while (m--) {
    int u, v, w;
    read(u), read(v), read(w);
    g[u][v] = min(g[u][v], (LL)w);
    cnt[u][v] = 1;
}
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        f[i][j] = g[i][j];
    }
}

for (int k = 1; k <= n; k++) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (g[i][j] > g[i][k] + g[k][j]) {
                g[i][j] = g[i][k] + g[k][j];
                cnt[i][j] = cnt[i][k] * cnt[k][j] % mod;
            } else if (g[i][j] == g[i][k] + g[k][j]) {
                cnt[i][j] = (cnt[i][j] + cnt[i][k] * cnt[k][j] % mod) % mod;
            }
        }
    }
    for (int i = 1; i < k; i++) {
        if (res > f[k][i] + g[i][k]) {
            res = f[k][i] + g[i][k];
            sum = cnt[i][k];
        } else if (res == f[k][i] + g[i][k]) {
            sum = (sum + cnt[i][k]) % mod;
        }
    }

}
```

30. 给定你 n 个数，你可以选择任意一个子集设 $\sum_{i\in S} a_i$ ，问最小不能被凑出的正整数是啥?

    我们考虑先对 n 个数排序，那么我们记 x，表示现在我们 $[0,x]$ 的数都可以被凑出，那么对于 ai

    若 $a_i > x+1$，说明 $x+1$ 是不能被凑出来的最小非负整数，因为此时前面所有数加起来都 $< x+1$，且 $a_i > x+1$，说明后面的

    数都要 $> x+1$，故我们无论选什么子集都不可能凑出 $x+1$。

    否则 $a_i \le x+1$，此时可以把 a_i 加入 x，表示我们用前 i 个数，能凑出 $[1, x+a_i]$ 的数。

    同时我们得到一个 trick：

    对于一个集合 S，其和为 x，且这个集合中选任意一个子集都可以凑出 $[0,x]$ 的所有数，若要加入新的数 a 加入到集合

    当且仅当，$a \le x+1$。

31. 期望线性性质：$E[\sum_i x_i] = \sum_i E[x_i]$。

32. 一类区间的贪心问题，通常按左端点或者右端点考虑，然后如果从前往后做，前面的已经满足，那我应该贪心的往后放,使得对后面影响尽可能大。（相当于在已经满足前面的条件的时候，尽可能扩大对后面还未考虑的影响，这样可以贪心的使得我们最后的总代价变小）

33. 树的各种性质：
    - 树的直径不是唯一的。
    - 树直径的两个端点必然是树的两个叶子节点。
    - 如果树的直径大小为 D，则有：

      若 D 为偶数，则所有直径必然汇聚在直径的中点处。

      若 D 为奇数，则所有直径必然汇聚于中间边 (u, v) 。
    - 某个点是树的重心当且仅当它的最大子树大小不大于整棵树大小的一半。
    - 树中所有点到某个点的**距离和**中，到重心的距离和是最小的；如果有两个重心，那么到它们的距离和一样。反过来，距离和最小的点一定是重心。
    - 树**至多有两个**重心。如果树有两个重心，那么它们**相邻**。此时树一定有**偶数**个节点，且可以被划分为两个大小相等的分支，每个分支各自包含一个重心。
    - 往树上增加或减少一个**叶子**，如果原节点数是奇数，那么重心可能**增加**一个，原重心**仍是重心**；如果原节点数是偶数，重心可能**减少**一个，另一个重心**仍是重心**。
    - 把两棵树通过一条边相连得到一棵新的树，则新的重心在**较大**的一棵树一侧的**连接点**与**原重心**之间的**简单路径**上。如果两棵树大小一样，则重心就是两个连接点。

```
找重心
int n, sz[MAXN], mss[MAXN]; // n：总结点数（请从外部传入），sz：树的大小，mss：最大子树大小
vector<int> ctr; // 重心
void dfs(int p, int fa = 0) // 找重心
{
    sz[p] = 1, mss[p] = 0;
    for (auto [to, w] : edges[p])
        if (to != fa)
        {
            dfs(to, p);
            mss[p] = max(mss[p], sz[to]);
            sz[p] += sz[to];
        }
    mss[p] = max(mss[p], n - sz[p]);
    if (mss[p] <= n / 2) ctr.push_back(p);
}
```

34. 博弈论考虑 SG 函数打表，或者记忆化 DP 打表，尝试找规律：

- 例如 $x\%(a+b), x\%(a-b), x\%a, x\%b$ 等规律
- 还有比如二进制的规律，上下取整，奇偶分类，特判情况等等。

35. 对于一类需要快速求出某个数左边前 k 个数，或者后边前 k 个数，且支持删除的题，我们可以采用**双向链表**维护。

    例如求所有 $\geq k$ 的连续子区间的第 k 大的和，我们就可以从小到大枚举值，看看这个数作为第 k 大对答案产生的贡献，

然后记录值的位置，利用链表实现向左和向右查找 k 个数。

36. 对于一类询问区间 $[l,r]$ 中 $\geq x$ 的最小数，或者 $\leq x$ 的最大数的时候，我们可以考虑离线的做法。

    我们可以把所有询问和原数组，按值从大到小或者从小到大排序，用线段树维护一个 $[1,n]$ 的序列，初始都是正无穷 / 负无穷，当做到一个询问 x 的时候，由于我们已经排好序，那么此时序列中的所有数都是 $\geq x$ 的，我们只要查询 $[l,r]$ 的区间最小值即可。

37. 若 $\sum x \leq N$ ，**如果把一样的 x 放在一起考虑，那么序列中最多只有 $O(\sqrt{N})$ 种不同的取值。**

    常见的根号分治套路：若 $n \leq \sqrt{n}$ 可以暴力维护，若 $n > \sqrt{n}$，说明这样的情况个数是 $\dfrac{n}{\sqrt{n}} = \sqrt{n}$ 级别

    这种情况我们可以采取打标记，或者其它维护方法，当询问答案的时候才去暴力计算这类情况的贡献。

38. $lowbit(x) \mid x$，即 $lowbit(x)$ 是能够整除 x 的最大的 2 次幂（$lowbit(x)$ 一定是 x 的一个因子）。

39. 当题目出现**只能从某个方向到某个方向，或者只能走到严格大于它的位置**，这种时候我们通常可以往 DP 考虑，只要按照位置顺序，或者权值顺序，就可以让转移具有拓扑序，就可以转化成 DP 问题了。

40. 对于一类推公式的期望，且具有最优决策的问题，例如此类式子：$E = p \times \sum_{i=1}^{n} min(A,B)$ 啥的，我们通常是会存在一个决策点 k，使得对于 $i \leq k$ 都是第一种决策，$i > k$ 则是第二种决策，这种式子一般具有凹凸性，可以考虑三分求出这个决策点。

41. 对于两球相撞方向交换的题，有一个经典的 trick：

- 碰撞看成穿越，不在具体关注每个小球的具体运动轨迹，只关系最终位置。
- 由于实际小球不会穿越，那么我们发现运动过程中小球的编号相对顺序是不变的，故我们只要把

最后所有位置算出来，从小到大排序，初始小球也从小到大排序，然后一一对应，就可以找到最终每个小球的位置。

42. 对于一类值域 $\leq 10^6$ 的题，可以想到在值域上 DP。

43. 对于一些数不能共存的题，例如 i 和 $i-1$ ，i 和 $2 \times i$ 不能同时存在，

    要想到一些经典的 DP 模型：状态机 DP，或者建图转成树上问题。

44. 对于一类和时间序有关，例如覆盖问题：后面的会覆盖前面的，要想**到逆向思维**，倒过来做，这样第一次枚举到的就是最终状态，还有一些正着不好维护的题，倒着做也许会变得很好维护。

45. 对于一类计数问题，但范围特别大，这时候要想想有没有什么性质，区间的延展性，策略变换关键点，

    使得对于某个区间 $[l, r]$ ，当 l 合法的时候，整个区间都是合法的，这样就可以做到快速计数。

46. 对于一类 swap / 修改 相邻位置，或者修改某一个点，要想到前后缀分解，提前预处理一些不变的地方，然后枚举修改的位置，把前后缀和修改的位置合并。

47. 一些思维题，尝试先从极端或者特殊情况思考，找寻解题的突破口。

48. 要求前缀中前 k 大的和，可以采用反悔贪心的堆来实现，先加入新数，如果堆大小 $> k$ 就去掉已经选择的最小的那个。

49. 一类要求操作数最小的思维题，我们尝试**考虑哪些情况是一定是必须要操作的**，得到这个**操作的下界**，

    此时多半答案就是这个下界，因为我们一般都可以构造一种方法使得操作答案取到这个**下界**。

50. 基环树森林特征：

- n 个点 n 条边的无向图。
- n 个点的有向图，且每个点的出度都是 1。

  每个环构成了一个强连通分量，树链上是一个拓扑序。

50. 对于一个长度为 n 的排列，一些置换问题，$i \to p_i$ 连边。一定形成若干个环，大小设为 $k_1, k_2, \ldots, k_m$

    那么其对于一个 $\{1, 2, 3\ldots, n\}$ 的排列，都不停的向 p_i 传球，其最少操作几轮可以回到初始状态。

    我们发现一个环至少需要 k 的轮次，故我们得到最小操作轮数即为：$lcm(k_1, k_2, .., k_m)$。

51. 对于一些置换，变换问题，想到 $i \to p_i$ 连边转化为图论问题。

52. 所有点的出度或者入度均为 1，或者 n 个点 n 条边，此时图肯定是一个基环树森林。

53. 对于给定字符串 S，每次可以交换相邻两个字符，问你最少几次操作可以变成字符串 T。

- 如果 S 里面没重复字符，这就是一个逆序对的经典 trick，把字符串 S 在 T 的位置映射找到，然后算逆序数即可。
- 如果 S 里面有重复字符，那么就采用如下贪心做法：

  按 $1 \le i \le n$ 的顺序遍历，对于一个位置 i，在 S 串中找到第一个 j 满足 $i \le j \le n$ 且 $S[j] = T[i]$，然后把 $S[j]$ 不断相邻交换

  到位置 i，期间统计操作次数。

54. 给定点 (x_1, y_1) 要求走到 (x_2, y_2)，每次只能往上下左右一个方向走一步，问有多少种方案。

    由于上下左右这样两维不是独立的，考虑经典的映射转化：

我们设 $u_x = x_1 + y_1$，$u_y = x_1 - y_1$，那么我们发现走法就会变成每次只会走以下四种情况：

$(-1,1),(-1,-1),(1,1),(1,-1)$，发现每次两维都在变化，所以独立了，我们可以分别求解两维的方案如何乘起来。

问题变成求 (x_1+y_1, x_1-y_1) 走到 (x_2+y_2, x_2-y_2) 的方案。

记 $f(n,m)$ 表示从 0 走 n 步之后到 m 的方案数，由组合数学已知：

$$f(n,m) = \begin{cases} C(n, \frac{(n+|m|)}{2}) & (n+|m|) \mod 2 = 0 \\ 0 & others \end{cases}$$

55. 给定 n 个点的度数限制，请你添加 $n-1$ 条边，使得他们满足度数限制，且构成一颗树。

    考虑度数序列能构造成树的充要条件：$\sum d = 2 \times n - 2$。

    如果 $\sum d \neq 2 \times n - 2$ 则无解，否则按如下构造方法：

    - 先把所有 $d_i \geq 2$ 的点，连成一条链。
    - 然后把所有 $d_i = 1$ 的点，挂在这些 $d_i \geq 2$ 的点上。

    注意如果没有 $d_i \geq 2$ 的点，此时必然只有两个单点，只要连接即可。

56. 对于一类集合合并，集合分裂的问题，要想到启发式合并（分裂），其复杂度都是 $O(n \log n)$ 的

    其思想就是每次遍历集合大小较小的一部分，即保证每次合并或者分裂的复杂度都是 $O(\min(sz_1, sz_2))$。

57. 一类较典的二分答案题型：

    - 最大平均数。
    - 最大中位数。
    - 01 分数规划，即形如 $\dfrac{\sum x}{\sum y}$ 的式子求最大值，或者最小值。
    - 第 k 小（大）。
    - 最大值最小，最小值最大。
    - 至少第几次操作之后满足 $\ldots$ 条件。
    - $\ldots.$（只要满足二段性都可以二分答案）。

58. 当 n 很大的时候，一类有解问题，要想到鸽笼原理是否成立，这类问题常常可以暴力，常见的：

    - 选一段数凑出 k 的倍数，显然 $n \geq k$ 的时候，必然有解。
    - 例如选某些数对，满足某些条件，什么什么不重复。

    鸽笼原理的核心就是，当基数很大的时候，某些东西会重复，所以只要小范围暴力即可。

59. 欧拉函数 $\phi(n)$，对于 n 一直做 $\phi(\phi(\ldots\phi(n)))$，那么大约 $\log$ 次之后 n 就会变成 1。

60. 一个字符串 S 的所有 Border 构成了一个border树，有 $n+1$ 个节点：$0,1,2,\ldots,n$，每个节点 i 的父亲为 $next[i]$。

    Border 的性质：

- 如果 p 是字符串的一个周期，那么 $|s|-p$ 是字符串的一个 Border (我们这里称 Border 是长度)。
- 求字符串所有 Border $\iff$ 求字符串所有前缀的最大 Border。只要一直跳 nxt 数组即可：

```
int p = nxt[n];
while (p) p = nxt[p];
```

- 一个串的 border 数量是 $O(n)$ 级别的，它们组成了 $\log(n)$ 个等差数列。
- 若 p,q 都是字符串的周期，那么 $\gcd(p,q)$ 也是字符串的周期。

Border 树的性质：

- 建树方式：$i \to nxt[i]$ 连边。
- 每个前缀 $pre[i]$ 的所有 Border 为节点 i 到根的链。
- 有哪些前缀有长度为 x 的 Border，相当于求 x 的子树大小。
- 求两个前缀的公共 Border 等价于求两个节点的 LCA。

61. 对于一类枚举 l，确定区间 $[l,l+s]$ 的题，考虑 l 的关键点其实只落在 a_i, a_i-s 这一共 $2n$ 个点。

62. 对于一类每次删除两个不同的数，问你最后数组长度最短是多少。

    要从**出现次数最大的元素**切入，记其出现次数为：mx，若 $mx > n-mx$，那么说明次数最大的都抵消不完，

    最后必然剩下：$2mx-n$。

    否则我们必然可以让数组消到只剩 1 或者 0，这取决于 n 的奇偶性。

    经典题：[C - Removal of Unattractive Pairs]，[2856. 删除数对后的最小数组长度]

63. 对于某些题，我们不要太在意复杂的过程，直接关注结果，可能会有突破口。

    例如：有 n 种颜色的球，每种分别有 a_i 个，每次你可以任意取 k 个球，问你最终颜色种类数最少是多少？

    我们直接做显然很复杂，我们考虑末态，由于我们每次都是取 k 个，那么我们最后必然剩下：$\sum a_i \ mod \ k$ 个球。

    那么我们反过来想，我们可不可以把问题变成，相当于我们从这 $\sum a_i$ 个球中，我先选出那些最终留下的球。

    那么问题就变成了贪心问题，我们肯定先挑 a_i 最多，这样剩下种类的 a_i 就能被全部拿掉。

64. 对于一些操作，或者数据范围，要观察敏感：
    - $\lfloor \frac{n}{k} \rfloor$，相当于 k 进制下的右移一位。
    - 每次选一个 k，让 n 变成 $nk+x$，其中 $0 \le x < k$。相当于 k 进制下的左移一位，然后随意添加这位的数字。
    - 数组的价值都是 k^m。
    - 注意进制的位权表示性质：若 $b_m > b_{m-1} > \ldots > b_0$，那么必然有 $k^{b_m} > k^{b_{m-1}} + k^{b_{m-2}} + \ldots + k^{b_0}$
    - $n = ax+b$，其中 $(b<a)$，那么这些数都有一个共同点就是 $\mod a$ 的余数都是 b。

65. 判断 $[l,r]$ 内是否存在 x 的倍数：
    - 其实我们只要看 $\geq l$ 的第一个 x 的倍数是否 $\leq r$ 即可。
    - 或者我们看 $l\ \%\ x$ ，是否是 0，如果不是就看 $x-l\ \%x$ 和 $r-l$ 的大小关系，即看还要过多几个数才到下一个 x 的倍数。

66. 给定一棵树，求 $\sum_{i=1}^{n}\sum_{j=i}^{n}I(i,j)$，或者求 $\sum_{i=1}^{n}\sum_{j=i}^{n}P(i,j)$

    其中 $I(i,j)$ 表示树上两点 i,j 路径上**边权**的最大值和最小值之差。

    其中 $P(i,j)$ 表示树上两点 i,j 路径上**点权**的最大值和最小值之差。

    十分经典的 trick，**排序+并查集**。
    - 不难发现可以最大值，最小值分开来算。
    - 对于边权：以最求最大值为例，按边权从小到大排序，对于一条边 (u,v) 那么所有和 u 在一个集合，和 v 在一个集合的点，两两之间构成的路径的最大值就是当前 (u,v) 的边权，算贡献：$sz_u\times sz_v\times w_{u,v}$ 即可。
    - 对于点权：以最求最大值为例，按点权从小到大排序，然后每次遍历所有 u 的邻边，若 v 已经在前面被处理过了，说明此时 v 所在集合的所有点都到 u 所在集合所有点构成路径的最大值就是当前点 u 的权值，算贡献即可。

67. 曼哈顿距离与切比雪夫距离的转化：

    $|x_i-x_j|+|y_i-y_j|\iff\max(|x'_i-x'_j|,|y'_i-y'_j|)$

    其中 $(x',y')=(x-y,x+y)$

    若 (x,y) 是曼哈顿的坐标，则其可以映射到 $(x-y,x+y)$

    若 (x,y) 是切比雪夫的坐标，则其可以映射到 $(\frac{x-y}{2},\frac{x+y}{2})$

68. 不同数据范围下的，因子个数，质因子个数分布：

$n\leq$	10^1	10^2	10^3	10^4	10^5	10^6	10^7	10^8	10^9
$\max\{\omega(n)\}$	2	3	4	5	6	7	8	8	9
$\max\{d(n)\}$	4	12	32	64	128	240	448	768	1344
$n\leq$	10^{10}	10^{11}	10^{12}	10^{13}	10^{14}	10^{15}	10^{16}	10^{17}	10^{18}
$\max\{\omega(n)\}$	10	10	11	12	12	13	13	14	15
$\max\{d(n)\}$	2304	4032	6720	10752	17280	26880	41472	64512	103680

洛谷

69. 给定一个 n 个点的无向完全图，同时给定 m 条删除的边，求图中每个连通块的大小。
    - 观察到删边次数是 $O(m)$ 的，我们考虑 BFS，同时维护一个目前还没确定所在连通块的点。
    - 每次由一个点去扩展，它只有两种情况：要么把某个点加入到自己的连通块中，否则一定是遍历到了一条删除的边。
    - 不难发现只要维护一个 `set` 或者链表支持删除即可。

- 总的复杂度可以达到 $O(n+m)$。

对于一些连边，删边，发生连锁反应等问题，要想到用队列去扩展（BFS 的思想）。

70. 生成树计数：
    - 对于一个 n 个点的无向完全图，其生成树个数为 n^{n-2}。
    - 对于一个 n 个点的无向完全图，每条边在其生成树中的贡献均为：$\frac{n^{n-2}\times(n-1)}{\frac{n\times(n-1)}{2}}=2\times n^{n-3}$。
71. 组合数的一些性质：
    1. $C_n^m=\frac{n!}{m!(n-m)!}$ （$n\geq m$）。
    2. $C_n^m=C_{n-1}^m+C_{n-1}^{m-1}$。
    3. $C_n^m=C_n^{n-m}$。（对称性）
    4. $C_{m+r+1}^r=\sum_{i=0}^r C_{m+i}^i$。（利用递推式证明）
    5. $C_m^n\times C_n^r=C_m^r\times C_{m-r}^{n-r}$。
    6. 二项式定理：
       $\sum_{i=0}^m C_m^i=2^m$。
       $(x+y)^m=\sum_{i=0}^m C_m^i x^i y^{m-i}$。
    7. $C_m^0+C_m^2+\ldots+C_m^{m-[m\%2=1]}=C_m^1+C_m^3+\ldots+C_m^{m-[m\%2=0]}=2^{m-1}$。
    8. $n\times C_m^n=m\times C_{m-1}^{n-1}$。
    9. $\sum_{i=1}^n C_n^i\times i=n\times 2^{n-1}$。
    10. $\sum_{i=0}^n (C_n^i)^2=C_{2n}^n$。
    11. $\sum_{i=1}^n C_n^i\times i^2=n\times(n+1)\times 2^{n-2}$。
    12. 卢卡斯定理：$C_m^n\equiv C_{m/p}^{n/p}\times C_{m\%p}^{n\%p}\mod p$
72. 虚拟源点：
    - 优化多次枚举起点，算最短路。
    - 点权转边权：建立虚拟源点 0，点权 $a_i\iff 0\to i$ 连了一条边权为 a_i 的边。
    - 在某个集合中的点互相连边，连边数很多，可以考虑建立虚拟源点，让这些点和虚拟源点建立边，达到等价的效果。
73. 可撤销并查集：

    需要使用启发式合并，然后利用栈记录合并路径，撤销操作相当于出栈。

```
struct DSU {
    std::pair<int, int> stk[N];
    int top, f[N], sz[N];
    void init(int n) {
        top = 0;
        for (int i = 1; i <= n; i++) {
            f[i] = i;
            sz[i] = 1;
        }
    }

    int find(int x) {
        while (x != f[x]) x = f[x];
        return x;
```

```
    }

    void merge(int u, int v) {
        int pu = find(u), pv = find(v);
        if (pu == pv) return;
        if (sz[pu] > sz[pv]) std::swap(pu, pv);
        f[pu] = pv;
        sz[pv] += sz[pu];
        stk[++top] = {pu, pv};
    }

    void roll(int t) {
        while (top > t) {
            auto [pu, pv] = stk[top--];
            f[pu] = pu;
            sz[pv] -= sz[pu];
        }
    }

    bool same(int u, int v) {
        return find(u) == find(v);
    }
};
```

74. 给定一个序列，你可以用任意整数替换任何元素，请问最少替换几次可以使得数组变成严格递增数组？

    经典 tirck：考虑保留哪些子序列，若保留 $a_j, a_i (j < i)$，那么必然满足：$a_i - a_j \geq i - j$，即 $a_i - i \geq a_j - j$

    那么问题变成了求序列 $\{a_i - i\}$ 的最长非下降子序列。

75. 用 x 条直线覆盖 n 个点：

    首先选出 $x + 1$ 个点，其任意两个点构成的直线，必然会是这 x 条覆盖直线中的一条，否则必然矛盾，可以反证法证明。

    所以做法，就是递归，每次随便选 $x + 1$ 个点，枚举它们两两之间的直线，然后删去被这条直线覆盖的点，继续递归，复杂度是

    $O(CN)$，其中 C 为和 x 相关的常量。

76. 对于 $[l, r]$ 的最大 border 记作 f，$[l - 1, r + 1]$ 的最大 border 记作 g，必然有 $g \leq f + 2$。

注意到一段长为 n 的字符串掐头去尾后黄色的这段仍然是新字符串的 `border` 。

所以我们可以从中心往两边扩，利用递推的方式，这种复杂度用势能分析是 $O(n)$ 的。

77. 方程 $k_1a_1 + k_2a_2 + \ldots + k_na_n = m$ 存在一组整数解 $\iff \gcd(a_1, a_2, \ldots, a_n) \mid m$。
78. $a + b = a \oplus b + 2(a\&b) = a\&b + a|b$。

    $a \oplus b = a|b - a\&b$。
79. 若 $a \equiv b \pmod m$，$b \equiv c \pmod m$，则 $a \equiv c \pmod m$。

    若 $a \equiv b \pmod m$，$c \equiv d \pmod m$，则有：
    - $a \pm c \equiv b \pm d \pmod m$。
    - $a \times c \equiv b \times d \pmod m$。

    若 $a \equiv b \pmod m$，则 $ak \equiv bk \pmod{mk}$。

    若 $d \mid m$，则当 $a \equiv b \pmod m$ 成立时，有 $a \equiv b \pmod d$。

    若 $a \equiv b \pmod{m_i}, i = 1, 2, 3, \ldots, k$，则 $a \equiv b \pmod{\operatorname{lcm}(m_1, m_2, m_3, \ldots, m_k)}$。

    若 $d \mid a$，$d \mid b$，则当 $a \equiv b \pmod m$ 成立时，有 $\frac{a}{d} \equiv \frac{b}{d} \pmod{\frac{m}{\gcd(m,d)}}$。

    若 $a \equiv b \pmod m$，则 $\gcd(a, m) = \gcd(b, m)$。
80. 常见几何结论。
    - 已知正方形对角线两点 $A(x_a, y_a)$，$C(x_c, y_c)$

      可求另外两点：$B(\frac{(x_a+x_c)-(y_a+y_c)}{2}, \frac{(y_a+y_c)+(x_a-x_c)}{2})$，
      $D(\frac{(x_a+x_c)+(y_a+y_c)}{2}, \frac{(y_a+y_c)-(x_a-x_c)}{2})$。
    - 已知正方形邻边两点 $A(x_a, y_a)$，$B(x_b, y_b)$

      可求另外两点：$C(x_b + y_a - y_b, y_b + x_b - x_a)$，$D(x_a + y_a - y_b, y_a + x_b - x_a)$。